

GUÍA PASO A PASO DE INSTALACIÓN, CONFIGURACIÓN Y DESPLIEGUE COMPLETO

Proyecto 1 — Sistemas Operativos 1

Universidad de San Carlos de Guatemala

Estudiante: Tulio Jafeth Pirir Schuman

Carnet: 201700698

1. INSTALACIÓN DE INFRAESTRUCTURA DE BASE EN EL HOST (KVM / QEMU)

Ejecute los siguientes comandos en la máquina host para instalar el entorno de virtualización:

Bash

1. Actualizar repositorios del sistema host

```
sudo apt update && sudo apt upgrade -y
```

2. Instalación de paquetes de KVM, QEMU, libvirt y herramientas

```
sudo apt install -y qemu-kvm libvirt-daemon-system libvirt-clients bridge-utils virt-manager  
neofetch jq
```

3. Habilitar y arrancar el demonio de libvirt

```
sudo systemctl enable --now libvirtd
```

4. Agregar el usuario actual a los grupos kvm y libvirt

```
sudo usermod -aG kvm $USER
```

```
sudo usermod -aG libvirt $USER
```

5. Comprobar la aceleración por hardware de la CPU

```
kvm-ok
```

6. Verificar el estado de las VMs en KVM

```
virsh list --all
```

2. CONFIGURACIÓN E INSTALACIÓN DE VMS Y RUNTIMES

Paso 1: Configuración de VM3 (192.168.122.13) — Docker y Zot Registry

Inicie sesión en la VM3 y ejecute:

Bash

1. Instalar Docker Engine

```
sudo apt update
```

```
sudo apt install -y docker.io curl jq
```

```
sudo systemctl enable --now docker
```

2. Desplegar el registro Zot OCI con almacenamiento persistente en disco

```
sudo mkdir -p /var/lib/zot
sudo docker run -d --name zot --restart always -p 5000:5000 \
-v /var/lib/zot:/var/lib/zot \
ghcr.io/project-zot/zot-linux-amd64:latest
```

3. Verificar la respuesta del catálogo Zot

```
curl -s http://localhost:5000/v2/_catalog | jq .
```

Paso 2: Configuración de VM1 (192.168.122.11) — Containerd y nerdctl

Inicie sesión en la VM1:

Bash

1. Instalar Containerd y Go

```
sudo apt update
sudo apt install -y containerd golang-go curl jq
sudo systemctl enable --now containerd
```

2. Instalar nerdctl (CLI para containerd)

```
NERDCTL_VERSION="1.7.0"
wget
https://github.com/containerd/nerdctl/releases/download/v${NERDCTL_VERSION}/nerdctl-${
NERDCTL_VERSION}-linux-amd64.tar.gz
sudo tar Cxzf /usr/local/bin nerdctl-${NERDCTL_VERSION}-linux-amd64.tar.gz
rm nerdctl-${NERDCTL_VERSION}-linux-amd64.tar.gz
```

3. Compilar e integrar API1 en la imagen OCI

```
cd ~/apis/api1
CGO_ENABLED=0 GOOS=linux go build -o server main.go
```

```
sudo nerdctl run --name temp-api1 alpine:latest /bin/true
sudo nerdctl cp server temp-api1:/server
sudo nerdctl commit --change 'CMD ["/server"]' temp-api1 api1-201700698:v1
sudo nerdctl rm temp-api1
sudo nerdctl tag api1-201700698:v1 192.168.122.13:5000/api1-201700698:v1
sudo nerdctl push 192.168.122.13:5000/api1-201700698:v1 --insecure-registry
```

4. Compilar e integrar API2 en la imagen OCI

```
cd ~/apis/api2
CGO_ENABLED=0 GOOS=linux go build -o server main.go
```

```
sudo nerdctl run --name temp-api2 alpine:latest /bin/true
sudo nerdctl cp server temp-api2:/server
sudo nerdctl commit --change 'CMD ["/server"]' temp-api2 api2-201700698:v1
sudo nerdctl rm temp-api2
sudo nerdctl tag api2-201700698:v1 192.168.122.13:5000/api2-201700698:v1
sudo nerdctl push 192.168.122.13:5000/api2-201700698:v1 --insecure-registry
```

Desplegar contenedores con la red del Host (--net=host)
sudo nerdctl rm -f api1 api2

```
sudo nerdctl run -d --name api2 --net=host \
-e API1_URL="http://127.0.0.1:8081/health" \
-e API3_URL="http://192.168.122.12:8083/health" \
192.168.122.13:5000/api2-201700698:v1
```

```
sudo nerdctl run -d --name api1 --net=host \
-e API2_URL="http://127.0.0.1:8082/health" \
-e API3_URL="http://192.168.122.12:8083/health" \
192.168.122.13:5000/api1-201700698:v1
```

Paso 3: Configuración de VM2 (192.168.122.12) — Podman

Inicie sesión en la VM2:

Bash

1. Instalar Podman

```
sudo apt update
```

```
sudo apt install -y podman curl jq
```

. Navegar al proyecto y revisar Dockerfile multi-stage
cd ~/apis/api3

3. Construir, etiquetar y publicar imagen en Zot

```
podman build -t api3-201700698:v1 .
```

```
podman tag api3-201700698:v1 192.168.122.13:5000/api3-201700698:v1
```

```
podman push --tls-verify=false 192.168.122.13:5000/api3-201700698:v1
```

4. Desplegar contenedor

```
podman run -d --name api3 --replace -p 8083:8083 \
```

```
-e API1_URL="http://192.168.122.11:8081/health" \
```

```
-e API2_URL="http://192.168.122.11:8082/health" \
```

```
--tls-verify=false \
```

```
192.168.122.13:5000/api3-201700698:v1
```

3. SCRIPT DE PRUEBAS DE EVALUACIÓN Y VALIDACIÓN

Ejecute la batería de pruebas en **VM1** para validar la solución ante el auxiliar:

Bash

En VM1:

```
sudo ctr namespace ls
```

```
sudo systemctl status containerd
```

En VM2:
podman info

En VM3:
docker info

curl -s http://localhost:8081/health | jq .
curl -s http://localhost:8082/health | jq .
curl -s http://192.168.122.12:8083/health | jq .

curl -s http://localhost:8081/api1/201700698/call-api2 | jq .
curl -s http://localhost:8081/api1/201700698/call-api3 | jq .
curl -s http://localhost:8082/api2/201700698/call-api1 | jq .
curl -s http://localhost:8082/api2/201700698/call-api3 | jq .